

UNIVERSITÀ DEGLI STUDI DI SALERNO

Dipartimento di Informatica

Corso di Laurea Magistrale in Informatica

PENETRATION TESTING & ETHICAL HACKING

AI-driven automated penetration testing tools

DOCENTE

Prof. Arcangelo Castiglione

Università degli Studi di Salerno

STUDENTE

Costantino Paciello

Matricola: 0522502045

Anno Accademico 2025-2026

Abstract

Questa attività progettuale analizza l'affidabilità operativa di sei framework di penetration testing basati su agenti Large Language Model: PentestGPT, Strix, Shannon, BugTraceAI, CAI e PentAGI. La sperimentazione è stata condotta su un ambiente locale isolato, utilizzando OWASP Juice Shop come target controllato.

I framework PentestGPT, Strix, Shannon, PentAGI e CAI sono stati testati con i modelli open-source Qwen2.5-Coder 7B e 14B mediante Ollama. BugTraceAI è stato testato esclusivamente tramite OpenRouter, utilizzando il modello gratuito `cohere/north-mini-code:free`.

I risultati evidenziano che BugTraceAI-CLI ha prodotto due vulnerabilità confermate, grazie al meccanismo di validazione incrociata dei finding. CAI ha riconosciuto un directory listing pubblico nell'endpoint /ftp con entrambi i modelli Qwen2.5-Coder 14B e 7B, mediante analisi di evidenze raccolte manualmente dall'operatore. PentestGPT ha invece confermato, nella sessione guidata con entrambi i modelli, una vulnerabilità di information disclosure mediante l'accesso non autenticato a un documento confidenziale nella medesima directory. Strix, PentAGI e Shannon non hanno completato le fasi operative a causa di errori di tool-calling e validazione dell'output.

I risultati suggeriscono che, con modelli open-source locali di dimensione contenuta, l'autonomia completa dei framework agentici è attualmente limitata dalla capacità del modello di rispettare protocolli strutturati di interazione con gli strumenti.

Indice

Elenco delle Figure **iv**

Elenco delle Tabelle **vii**

1 Introduzione e Stato dell'Arte **1**

1.1 Introduzione 1

1.2 Dal Vulnerability Scanning Tradizionale agli Agenti LLM 2

1.2.1 Vulnerability Scanning Tradizionale 2

1.2.2 Limiti dell'Automazione Deterministica 2

1.2.3 L'Emergere degli LLM nel Penetration Testing 3

1.3 Architettura di PentestGPT 3

1.3.1 Origine e Motivazione 3

1.3.2 I Tre Moduli Fondamentali 4

1.3.3 Il Pentesting Task Tree (PTT) 4

1.3.4 Paradigma Human-in-the-Loop 5

1.3.5 Evoluzione verso l'Autonomia 5

1.4 Posizionamento nel Panorama Attuale 5

1.5 Obiettivi del Progetto 6

2 Architettura Comparata dei Sei Framework **7**

2.1 Introduzione 7

2.2 PentestGPT 9
2.3 Strix 9
2.4 Shannon 10
2.5 BugTraceAI 10
2.6 CAI 11
2.7 PentAGI 11
2.8 Confronto critico 12

3 Metodologia Sperimentale **13**
3.1 Ambiente hardware e software 13
3.2 Architettura di inferenza locale 14
3.2.1 Ollama come runtime LLM 14
3.2.2 Configurazione del modello 7B 14
3.2.3 Configurazione del modello 14B 14
3.2.4 Confronto tra le due configurazioni 15
3.3 Configurazione aggiuntiva di BugTraceAI tramite OpenRouter . . . 15
3.3.1 Motivazione della configurazione 15
3.3.2 Provider e modello utilizzati 15
3.3.3 Tentativo di configurazione di BugTraceAI con Ollama 17
3.4 Configurazione del target di test 17
3.4.1 Scelta di OWASP Juice Shop 17
3.4.2 Isolamento tramite Docker 17
3.4.3 Configurazione di rete 18
3.5 Procedura sperimentale 18
3.5.1 Criteri di classificazione dei risultati 19
3.5.2 Distinzione tra assenza di finding ed esecuzione fallita 19
3.6 Limitazioni metodologiche 20

4 Risultati Sperimentali e Discussione Critica **21**
4.1 Panoramica dei risultati 21
4.2 Analisi per framework 22
4.2.1 BugTraceAI-CLI 22
4.2.2 CAI 24

4.2.3 PentestGPT 26
4.2.4 Strix 32
4.2.5 PentAGI 33
4.2.6 Shannon 34
4.3 Confronto tra i modelli Qwen 36
4.3.1 Throughput e velocità 36
4.3.2 Aderenza al tool-calling 36
4.3.3 Capacità di self-correction 37
4.3.4 Nota metodologica sull'uso di OpenRouter 37
4.4 Discussione critica 38
4.4.1 Il limite architetturale dei modelli locali 38
4.4.2 L'inefficacia del self-correction e contextual forgetting 38
4.4.3 Il paradigma vincente allo stato attuale 39
4.5 Considerazioni etiche e di sicurezza 39
4.5.1 Autorizzazione e perimetro di test 39
4.5.2 Safety guardrail 39
4.5.3 Vettori d'attacco e rischi associati 40

5 Conclusioni e Sviluppi Futuri **41**

Bibliografia **42**

Elenco delle figure

3.1 Interrogazione dell'endpoint `/api/config` di BugTraceAI: mappatura esplicita del modello gratuito `cohere/north-mini-code:free` sui vari ruoli della pipeline agentica. 16

4.1 Dettaglio del finding validato per *Missing Rate Limiting*: correlazione tra l'ipotesi dell'agente e la verifica tramite template Nuclei specializzato. 23

4.2 Riepilogo dello stato della scansione via API (`/api/scans/1/status`): completamento al 100% con individuazione iniziale di 6 riscontri grezzi prima del filtraggio. 23

4.3 Log di esecuzione della pipeline di BugTraceAI-CLI: evidenza del passaggio sequenziale tra le fasi (Discovery, Strategy, Exploitation, Validation e Reporting) con binding persistente su `cohere/north-mini-code:free`. 24

4.4 Interfaccia web di OWASP Juice Shop: evidenza empirica della directory listing esposta sull'endpoint `/ftp` con file sensibili e di backup visibili. 25

4.5 Variabili d'ambiente e parametri operativi di CAI: evidenza dell'attivazione dei guardrail di sicurezza (`CAI_GUARDRAILS`) e del vincolo al modello locale. 25

4.6 Output del report di sintesi generato da CAI: scomposizione formale tra fatti osservati empiricamente, ipotesi operative e affermazioni prive di riscontro probatorio. 26
4.7 Inizializzazione di PentestGPT con i moduli di reasoning, generation e parsing agganciati a Ollama locale e generazione iniziale del Pentesting Task Tree. 27
4.8 PentestGPT con Qwen2.5-Coder 14B: il modello, ricevuto il banner HTTP, seleziona direttamente test di iniezione (XSS) ipotizzando parametri non ancora rilevati. 27
4.9 PentestGPT con Qwen2.5-Coder 7B: l'albero dei task evidenzia la tendenza a richiedere passaggi di identificazione delle versioni già completati dall'evidenza fornita. 28
4.10 PentestGPT nella sessione guidata (14B): aggiornamento del PTT con marcatura dell'endpoint `/ftp` e proposta del comando mirato per il download di `acquisitions.md`. 30
4.11 Conclusione della sessione guidata: l'albero dei task viene marcato interamente come completato, ma il modello dichiara l'avvenuta redazione del documento senza produrre il corpo del report finale in Markdown. 31
4.12 PentestGPT con Qwen2.5-Coder 7B in sessione guidata: evidenza di amnesia contestuale e rottura del protocollo di stato. 32
4.13 Collasso del tool-calling in Strix: il modello emette la struttura JSON nel testo libero anziché invocare la funzione API. 33
4.14 Dettagli della sessione interrotta in Strix: stallo operativo e consumo asimmetrico di token di contesto. 33
4.15 Fase di validazione preliminare (*preflight checks*) di Shannon: connessione all'endpoint Ollama e raggiungibilità del target completate con successo. 35
4.16 Log di esecuzione di Shannon con Qwen2.5-Coder 7B: fallimento sistematico (`OUTPUT_VALIDATION_FAILED`) nei nodi di validazione ed exploitation. 35

4.17 Verifica comparativa dell'anomalia di tool-calling su endpoint Ollama: sia Qwen2.5-Coder 7B che 14B serializzano il comando JSON nel campo `content` anziché invocare nativamente la funzione API. . . . 37

Elenco delle tabelle

2.1 Confronto architetturale dei sei framework analizzati. 8
2.2 Meccanismi di controllo e impatto sui falsi positivi. 12

3.1 Configurazione hardware e provider LLM utilizzati. 13
3.2 Confronto tra le configurazioni di inferenza dei due modelli Qwen. . 15

4.1 Risultati complessivi della sperimentazione su singola sessione, distinti per framework e provider LLM. 22
4.2 Confronto prestazionale tra Qwen2.5-Coder 7B e 14B. 36

CAPITOLO 1

Introduzione e Stato dell'Arte

1.1 Introduzione

Il penetration testing è una pratica fondamentale per la valutazione della sicurezza di sistemi informatici, applicazioni web e infrastrutture di rete. Tradizionalmente, questa attività richiede professionisti altamente specializzati, capaci di pianificare campagne di test, selezionare strumenti appropriati, interpretare risultati parziali e concatenare più vulnerabilità in percorsi di attacco realistici. Tale complessità ha storicamente limitato l'automazione completa del processo: gli strumenti classici sono efficaci nell'individuare singole debolezze note, ma faticano a ragionare sul contesto complessivo dell'attacco.

Negli ultimi anni, l'emergere dei *Large Language Model* (LLM) ha aperto nuove prospettive per l'automazione intelligente delle attività di sicurezza. A differenza degli scanner tradizionali, un LLM può interpretare output testuali, formulare ipotesi, suggerire comandi successivi e adattare la propria strategia alle informazioni raccolte durante l'esecuzione. Tuttavia, come evidenziato da Deng et al., gli LLM mostrano buone capacità su sotto-task isolati — come l'uso di strumenti, l'interpretazione degli output e la proposta di azioni successive — ma incontrano difficoltà nel mantenere una comprensione integrata dell'intero scenario di penetration testing [1].

1.2 Dal Vulnerability Scanning Tradizionale agli Agenti LLM

1.2.1 Vulnerability Scanning Tradizionale

Gli strumenti classici di *vulnerability scanning* si basano prevalentemente su firme, database di vulnerabilità note e regole deterministiche. Esempi rappresentativi includono Nmap per la discovery di servizi, Nikto per l'analisi di server web, OWASP ZAP per il testing dinamico delle applicazioni e Nuclei per l'esecuzione di template di vulnerabilità.

Questi strumenti presentano diversi vantaggi:

- Elevata ripetibilità e prevedibilità dei risultati.
- Bassa latenza e consumo computazionale contenuto.
- Facilità di integrazione in pipeline CI/CD.
- Copertura affidabile di vulnerabilità note e catalogate.

Tuttavia, il loro limite principale risiede nell'assenza di ragionamento contestuale. Uno scanner tradizionale può segnalare la presenza di un endpoint esposto o di un header di sicurezza mancante, ma non è in grado di valutare autonomamente se tale debolezza possa essere sfruttata in una catena d'attacco più complessa, né di adattare la propria strategia in base ai risultati parziali ottenuti.

1.2.2 Limiti dell'Automazione Deterministica

L'automazione tradizionale si basa su un modello a regole: se una condizione nota è soddisfatta, viene generato un alert. Questo approccio è efficace per vulnerabilità ben definite, ma presenta limiti significativi in scenari reali:

- Impossibilità di ragionare su dipendenze tra vulnerabilità.
- Difficoltà nell'interpretare output non strutturati.
- Assenza di capacità di pianificazione adattiva.

- Tendenza a generare falsi positivi in contesti applicativi complessi.
- Incapacità di formulare ipotesi su comportamenti anomali non previsti dalle regole.

Questi limiti hanno motivato l'interesse verso approcci più flessibili, capaci di combinare strumenti deterministici con capacità di ragionamento linguistico e pianificazione.

1.2.3 L'Emergere degli LLM nel Penetration Testing

I *Large Language Model* hanno introdotto un nuovo paradigma: la possibilità di interpretare linguaggio naturale, contesto tecnico e output di strumenti in modo unificato. Un agente basato su LLM può, in linea teorica:

- Leggere l'output di Nmap e dedurre la superficie d'attacco.
- Interpretare risposte HTTP e identificare pattern sospetti.
- Suggerire payload o comandi successivi.
- Mantenere una memoria del contesto di attacco.
- Adattare la strategia in base ai risultati intermedi.

Tuttavia, questa flessibilità introduce nuovi problemi: allucinazioni, perdita di contesto, errori di formato nelle chiamate agli strumenti e difficoltà nel mantenere coerenza su sessioni lunghe. Questi aspetti sono centrali nella presente sperimentazione, come discusso nel Capitolo 4.

1.3 Architettura di PentestGPT

1.3.1 Origine e Motivazione

PentestGPT è stato proposto da Deng et al. come risposta diretta ai limiti degli LLM nel penetration testing. Gli autori hanno osservato che, pur mostrando competenze su singole sotto-attività, i modelli linguistici tendono a perdere la visione d'insieme quando devono gestire sessioni complesse e multi-step [1].

La soluzione proposta consiste nel decomporre il processo di penetration testing in moduli specializzati, ciascuno responsabile di un sotto-task specifico, con meccanismi di interazione strutturata tra i moduli stessi.

1.3.2 I Tre Moduli Fondamentali

PentestGPT si basa su tre moduli auto-interagenti [1]:

1. **Reasoning Module** — responsabile della pianificazione strategica e dell'analisi del contesto.
2. **Generation Module** — incaricato di generare comandi, payload e azioni operative.
3. **Parsing Module** — dedicato all'interpretazione degli output degli strumenti e alla loro traduzione in informazioni strutturate.

Questa separazione delle responsabilità mitiga il problema della perdita di contesto: invece di affidare a un singolo modello l'intero flusso, ogni modulo opera su un sottoinsieme coerente di informazioni.

1.3.3 Il Pentesting Task Tree (PTT)

Un elemento architetturale centrale di PentestGPT è il **Pentesting Task Tree** (PTT): una rappresentazione gerarchica dello stato dell'attacco, che consente di tracciare obiettivi, sotto-obiettivi, azioni completate e risultati ottenuti [1], [6].

Il PTT funge da memoria strutturata della sessione, permettendo all'agente di:

- Mantenere coerenza tra le fasi.
- Riprendere sessioni interrotte.
- Prioritizzare le azioni successive.
- Ridurre la ridondanza nelle richieste al modello.

1.3.4 Paradigma Human-in-the-Loop

Nella sua formulazione originale, PentestGPT adotta un paradigma *human-in-the-loop*: l'operatore umano esegue i comandi suggeriti dal sistema, ne osserva gli output e li reinserisce nel flusso di interazione [1], [6].

Questo design riduce il rischio di esecuzione autonoma di azioni dannose, ma introduce anche un collo di bottiglia operativo: la velocità e la qualità del test dipendono strettamente dalla capacità dell'operatore di mediare tra il modello e l'ambiente target.

1.3.5 Evoluzione verso l'Autonomia

Le versioni più recenti di PentestGPT hanno introdotto una modalità agentica autonoma, basata su pipeline multi-stadio (recon $\rightarrow$ exploit $\rightarrow$ walkthrough per CTF; asset discovery $\rightarrow$ vulnerability identification $\rightarrow$ report per penetration test) [6]. In questa configurazione, l'agente può guidare l'esecuzione degli strumenti senza intervento umano continuo, avvicinandosi al paradigma dei framework *fully autonomous* analizzati in questo progetto.

1.4 Posizionamento nel Panorama Attuale

PentestGPT rappresenta un punto di transizione tra due paradigmi:

- Il modello **copilota interattivo**, in cui l'umano guida e valida ogni passo [1].
- Il modello **agente autonomo**, in cui il sistema pianifica ed esegue in modo indipendente [7], [8], [9].

Questa dicotomia è al centro della presente sperimentazione: i risultati mostreranno come i framework *fully autonomous* (Strix, Shannon, PentAGI) incontrino difficoltà significative con modelli locali di piccola dimensione, mentre gli approcci ibridi o con validazione a valle (BugTraceAI-WEB, CAI) dimostrino maggiore robustezza operativa [10], [11].

1.5 Obiettivi del Progetto

La sperimentazione ha come obiettivo principale la valutazione comparativa di sei framework di penetration testing basati su agenti LLM: PentestGPT, Strix, Shannon, BugTraceAI, CAI e PentAGI. Il target scelto per i test è OWASP Juice Shop, un'applicazione web volutamente vulnerabile sviluppata in Node.js, Express e Angular, che copre vulnerabilità relative all'intera OWASP Top 10 [2].

Sulla base di questo perimetro, gli obiettivi specifici del progetto si articolano nei seguenti punti:

1. **Analisi architetturale:** Analizzare l'architettura e il funzionamento dei sei framework selezionati.

2. **Setup dell'ambiente:** Configurare un ambiente di test locale, isolato e riproducibile.

3. **Esecuzione dei test:** Eseguire campagne di test su OWASP Juice Shop utilizzando modelli locali (Qwen2.5-Coder 7B e 14B) per PentestGPT, Strix, Shannon, PentAGI e CAI, e il provider OpenRouter per BugTraceAI-CLI.

4. **Valutazione e confronto:** Raccogliere e confrontare metriche di efficacia, precisione e affidabilità operativa.

5. **Analisi critica:** Discutere criticamente i limiti dell'autonomia completa con modelli open-source locali.

CAPITOLO 2

Architettura Comparata dei Sei Framework

2.1 Introduzione

I sei framework analizzati non condividono un'unica architettura né un medesimo paradigma operativo. Alcuni sistemi, come PentestGPT, sono stati concepiti come copiloti interattivi, in cui l'operatore umano mantiene un ruolo attivo nell'esecuzione dei comandi e nella validazione dei risultati [1], [6].

Altri, come Strix, Shannon e PentAGI, perseguono un obiettivo di autonomia più elevato: il sistema deve pianificare, invocare strumenti, interpretare i risultati e proseguire il test senza intervento continuo dell'utente [7], [8], [9]. Questa differenza architetturale ha un impatto diretto sui requisiti imposti al modello linguistico, in particolare per quanto riguarda l'aderenza ai formati strutturati e la capacità di effettuare chiamate corrette agli strumenti.

La Tabella 2.1 riassume il posizionamento dei sei framework rispetto ai principali criteri architetturali.

Tabella 2.1: Confronto architetturale dei sei framework analizzati.

Framework	Paradigma	Autonomia	Caratteristica distintiva
PentestGPT	Copilota interattivo	Human-in-the-loop	Pipeline multi-modulo e Pentesting Task Tree [1], [6]
Strix	Agente autonomo	Fully autonomous	Agenti AI per test dinamici e validazione tramite PoC [7]
Shannon	Agente autonomo white/grey-box	Fully autonomous	Analisi del codice sorgente combinata con exploit dinamici [8]
BugTraceAI	Piattaforma web / scanner AI	Fully autonomous	Validazione incrociata dei finding e riduzione dei falsi positivi [10]
CAI	Framework AI con guardrail	Human-in-the-loop	Interazione guidata e attenzione a sicurezza IT/OT [11]
PentAGI	Orchestratore multi-agente	Fully autonomous	Orchestrazione di agenti per task complessi di penetration testing [9]

2.2 PentestGPT

PentestGPT si basa sull'architettura multi-modulo già descritta nella Sezione 1.3, articolata nei moduli di reasoning, generation e parsing e centrata sul Pentesting Task Tree (PTT). Nella configurazione classica opera secondo un paradigma human-in-the-loop; le versioni più recenti introducono una modalità agentica autonoma basata su pipeline multi-stadio (recon, exploit, report generation) [1]. Questa struttura riduce il rischio di perdita di contesto, uno dei problemi principali nell'utilizzo di LLM in attività multi-step.

Nella configurazione classica, PentestGPT opera secondo un paradigma *human-in-the-loop*: l'operatore esegue i comandi suggeriti, osserva gli output e li reinserisce nel sistema [6]. Le versioni più recenti introducono inoltre una modalità agentica autonoma, basata su pipeline multi-stadio quali recon, exploit e report generation [6].

2.3 Strix

Strix è un framework open-source composto da agenti AI autonomi per il penetration testing. Il sistema è progettato per eseguire test dinamici sulle applicazioni, individuare vulnerabilità e produrre proof-of-concept per validare i risultati [7].

A differenza di PentestGPT nella sua formulazione interattiva, Strix punta a un'esecuzione maggiormente autonoma. L'agente deve quindi essere in grado di selezionare gli strumenti appropriati, interpretarne gli output e proseguire il flusso operativo senza richiedere l'intervento continuo dell'utente.

Questa autonomia richiede un'interazione rigorosa con le interfacce di tool-calling. Il modello deve produrre chiamate strutturate, rispettare gli schemi attesi dall'orchestratore e gestire correttamente gli stati della sessione. Durante la sperimentazione, questa esigenza si è rivelata particolarmente critica quando Strix è stato configurato con modelli locali di dimensione contenuta.

2.4 Shannon

Shannon è un agente autonomo per il penetration testing di applicazioni web e API. La sua architettura combina analisi del codice sorgente, identificazione di potenziali vettori d'attacco ed esecuzione di exploit dinamici contro l'applicazione in esecuzione [8].

Il framework adotta un approccio white-box o grey-box: oltre all'interazione con l'applicazione in runtime, può analizzare il codice sorgente per individuare percorsi di attacco potenziali. Shannon include inoltre meccanismi di validazione basati su proof-of-concept: solo le vulnerabilità per le quali è stato prodotto un PoC funzionante vengono incluse nel report finale [8].

L'architettura multi-agente di Shannon prevede la collaborazione di più componenti specializzati, con fasi parallele di analisi delle vulnerabilità ed exploitazione [8]. Questa complessità rende il framework potenzialmente efficace, ma aumenta anche i requisiti di affidabilità del modello linguistico, soprattutto in termini di rispetto dei formati di output e di coordinamento tra agenti.

2.5 BugTraceAI

BugTraceAI è una piattaforma self-hosted per bug bounty e penetration testing che combina agenti AI con strumenti di sicurezza deterministici [10]. Il suo approccio si basa sull'idea che l'AI debba formulare ipotesi e priorità, mentre gli strumenti tradizionali eseguano controlli concreti, validino le evidenze e producano report ispezionabili [10].

Questa architettura introduce un meccanismo di validazione a valle dei finding generati dal modello. Anche qualora l'LLM produca un'ipotesi errata o un falso positivo, il sistema può filtrare il risultato prima che venga incluso nel report finale.

Nel contesto della sperimentazione, questa caratteristica ha rappresentato uno dei principali punti di forza di BugTraceAI: la capacità di scartare automaticamente falsi positivi ha migliorato la precisione del report, senza richiedere un intervento manuale dell'operatore.

2.6 CAI

CAI, sviluppato da Alias Robotics, è un framework AI orientato alla cybersecurity, con particolare attenzione anche al dominio IT/OT [11]. A differenza dei sistemi completamente autonomi, CAI adotta un paradigma più interattivo, in cui l'operatore umano mantiene un ruolo di supervisione e autorizzazione.

Il framework consente all'agente di dedurre informazioni dal contesto, interpretare metadati e proporre azioni successive. Tuttavia, questa flessibilità può aumentare il rischio di inferenze non supportate da evidenze concrete. Durante la sperimentazione, ad esempio, il modello ha mostrato una tendenza ad allucinare dettagli non verificati, come la presenza di un pannello amministrativo dedotto esclusivamente da un header HTTP.

Nonostante ciò, il paradigma human-in-the-loop di CAI consente all'operatore di intercettare e correggere queste derive prima che producano un impatto sul risultato finale.

2.7 PentAGI

PentAGI è un sistema multi-agente completamente autonomo progettato per l'esecuzione di task complessi di penetration testing [9]. L'architettura si basa sull'orchestrazione di più agenti, ciascuno responsabile di una parte del flusso operativo.

Il sistema richiede che il modello linguistico produca risposte strutturate, tipicamente in formato JSON, per invocare correttamente gli strumenti e comunicare con l'orchestratore. Questa esigenza rende PentAGI particolarmente sensibile alla qualità del tool-calling del modello utilizzato.

Durante la sperimentazione, l'orchestratore ha mostrato criticità significative quando è stato configurato con modelli locali di piccola dimensione: il modello tendeva a produrre testo discorsivo anziché chiamate strutturate, causando errori di parsing e interruzioni del flusso operativo.

2.8 Confronto critico

L'analisi architetturale evidenzia una dicotomia fondamentale tra i sei framework:

- **Approcci con supervisione o validazione**: PentestGPT, CAI e BugTraceAI introducono meccanismi che riducono l'impatto degli errori del modello, rispettivamente tramite intervento umano, supervisione operativa o validazione automatica dei finding.
- **Approcci completamente autonomi**: Strix, Shannon e PentAGI delegano all'agente l'intero flusso operativo, aumentando il fabbisogno di affidabilità nel tool-calling e nella gestione dello stato.

Questa distinzione ha un impatto diretto sui risultati sperimentali. I framework completamente autonomi risultano più sensibili agli errori di formato, alle allucinazioni e alla degradazione del contesto nelle sessioni lunghe. Al contrario, i sistemi che prevedono validazione o supervisione riescono a contenere meglio questi problemi, anche quando il modello linguistico non è perfettamente affidabile.

Framework	Meccanismo di controllo	Effetto sui falsi positivi
PentestGPT	Supervisione umana	Riduzione tramite revisione manuale
CAI	Human-in-the-loop	Riduzione tramite autorizzazione operatore
BugTraceAI	Validazione incrociata	Riduzione automatica
Strix	Autonomia completa	Rischio elevato in caso di errori del modello
Shannon	Validazione tramite PoC	Riduzione se il PoC viene generato
PentAGI	Autonomia completa	Rischio elevato in caso di errori del modello

Tabella 2.2: Meccanismi di controllo e impatto sui falsi positivi.

CAPITOLO 3

Metodologia Sperimentale

3.1 Ambiente hardware e software

Le sperimentazioni sono state condotte su una workstation locale con la seguente configurazione:

Tabella 3.1: Configurazione hardware e provider LLM utilizzati.

Componente	Specifiche
CPU	AMD Ryzen 5 5600X (6c/12t)
GPU	AMD Radeon RX 6650 XT (8 GB VRAM)
RAM	32 GB DDR4
Sistema operativo	Windows 11 con WSL2 (Ubuntu 24.04)
Runtime LLM locale	Ollama
Provider cloud	OpenRouter (solo per BugTraceAI)
Virtualizzazione	Docker Desktop su WSL2
Target	OWASP Juice Shop

La presenza di una GPU con 8 GB di VRAM ha influito in modo significativo sulla strategia di inferenza. Il modello da 7B parametri è stato eseguito interamente in

VRAM, mentre il modello da 14B parametri ha richiesto un meccanismo di offloading ibrido tra GPU e memoria di sistema.

3.2 Architettura di inferenza locale

3.2.1 Ollama come runtime LLM

L'inferenza dei modelli locali è stata gestita tramite Ollama, un runtime open-source che consente di eseguire modelli linguistici localmente e di esporli tramite un'API locale [3], [12]. Ollama valuta automaticamente la memoria disponibile e decide come distribuire i layer del modello tra GPU e CPU [13], [14].

Quando un modello entra interamente nella VRAM della GPU, Ollama lo carica con una configurazione `100% GPU`, ottenendo generalmente le migliori prestazioni [13]. Quando il modello non entra interamente nella VRAM disponibile, il runtime effettua un offloading parziale: una parte dei layer viene caricata sulla GPU, mentre il resto viene eseguito sulla CPU utilizzando la memoria di sistema [13].

3.2.2 Configurazione del modello 7B

Il modello `qwen2.5-coder:7b` è stato eseguito interamente in VRAM sulla GPU AMD Radeon RX 6650 XT. Questa configurazione ha permesso di ottenere un throughput superiore rispetto alla configurazione ibrida, riducendo la latenza di inferenza e il traffico dati tra CPU e GPU.

La scelta del modello 7B è stata motivata dalla necessità di testare uno scenario in cui l'intero modello risiedesse nella VRAM, massimizzando la velocità di generazione.

3.2.3 Configurazione del modello 14B

Il modello `qwen2.5-coder:14b` non è stato in grado di risiedere interamente nella VRAM disponibile. Per questo motivo, Ollama ha adottato una strategia di offloading ibrido, caricando una parte del modello sulla GPU e allocando il resto nella RAM di sistema.

Questa configurazione ha permesso di eseguire un modello con maggiore capacità di ragionamento, ma ha introdotto un overhead significativo dovuto al trasferimento di dati tra memoria di sistema e GPU. Il risultato è stato un throughput inferiore rispetto al modello 7B, pari a circa 10,86 token/s contro 29,87 token/s.

3.2.4 Confronto tra le due configurazioni

Tabella 3.2: Confronto tra le configurazioni di inferenza dei due modelli Qwen.

Metrica	Qwen2.5-Coder 7B	Qwen2.5-Coder 14B
Modalità di esecuzione	Interamente in VRAM	Offloading ibrido GPU/RAM
Throughput	∼29,87 token/s	∼10,86 token/s
Velocità relativa	∼3× più veloce	Riferimento
Capacità di ragionamento	Limitata dalla dimensione del modello	Potenzialmente superiore
Aderenza al tool-calling	Instabile sotto stress	Instabile nelle sessioni lunghe

I modelli della serie Qwen2.5-Coder sono disponibili in diverse dimensioni, tra cui 7B e 14B parametri, e sono ottimizzati per attività di generazione, ragionamento e correzione del codice [15], [16].

3.3 Configurazione aggiuntiva di BugTraceAI tramite OpenRouter

3.3.1 Motivazione della configurazione

Oltre alle sperimentazioni condotte con modelli locali tramite Ollama, è stata effettuata una configurazione aggiuntiva di BugTraceAI utilizzando OpenRouter come provider cloud. Questa configurazione è stata adottata esclusivamente per il framework BugTraceAI e non per gli altri tool analizzati.

3.3.2 Provider e modello utilizzati

Il provider configurato è stato `openrouter-v2`, con il seguente endpoint:

```
https://openrouter.ai/api/v1/chat/completions
```

Il modello gratuito selezionato per i ruoli testuali principali è stato:

```
cohere/north-mini-code:free
```

Tale modello è stato applicato ai ruoli della pipeline BugTraceAI, visibili nella Figura 3.1

La validità della chiave API e la raggiungibilità del provider sono state verificate tramite l'endpoint di test di BugTraceAI, che ha restituito una conferma positiva della configurazione.

```
curl -fsS http://localhost:8000/api/config \
  | python3 -c '
import json, sys
cfg = json.load(sys.stdin).get("config", {})
wanted = [
    "DEFAULT_MODEL",
    "CODE_MODEL",
    "ANALYSIS_MODEL",
    "PRIMARY_MODELS",
    "WAF_DETECTION_MODELS",
    "MUTATION_MODEL",
    "SKEPTICAL_MODEL",
    "ANALYSIS_PENTESTER_MODEL",
    "ANALYSIS_BUG_BOUNTY_MODEL",
    "ANALYSIS_AUDITOR_MODEL",
    "ANALYSIS_RED_TEAM_MODEL",
    "ANALYSIS_RESEARCHER_MODEL",
    "LONEWOLF_MODEL",
    "LONEWOLF_ENABLED",
    "REPORTING_MODEL",
    "REPORTING_FAILOVER_ENABLED",
    "MIN_CREDITS",
    "VALIDATION_VISION_ENABLED"
]
for key in wanted:
    print(f"{key} = {cfg.get(key)}")
'
DEFAULT_MODEL = cohere/north-mini-code:free
CODE_MODEL = cohere/north-mini-code:free
ANALYSIS_MODEL = cohere/north-mini-code:free
PRIMARY_MODELS = cohere/north-mini-code:free
WAF_DETECTION_MODELS = cohere/north-mini-code:free
MUTATION_MODEL = cohere/north-mini-code:free
SKEPTICAL_MODEL = cohere/north-mini-code:free
ANALYSIS_PENTESTER_MODEL = cohere/north-mini-code:free
ANALYSIS_BUG_BOUNTY_MODEL = cohere/north-mini-code:free
ANALYSIS_AUDITOR_MODEL = cohere/north-mini-code:free
ANALYSIS_RED_TEAM_MODEL = cohere/north-mini-code:free
ANALYSIS_RESEARCHER_MODEL = cohere/north-mini-code:free
LONEWOLF_MODEL = cohere/north-mini-code:free
LONEWOLF_ENABLED = False
REPORTING_MODEL = cohere/north-mini-code:free
REPORTING_FAILOVER_ENABLED = False
MIN_CREDITS = 0.0
VALIDATION_VISION_ENABLED = False
```

Figura 3.1: Interrogazione dell'endpoint `/api/config` di BugTraceAI: mappatura esplicita del modello gratuito `cohere/north-mini-code:free` sui vari ruoli della pipeline agentica.

3.3.3 Tentativo di configurazione di BugTraceAI con Ollama

È stato effettuato un tentativo di configurazione di BugTraceAI con modelli locali Qwen2.5-Coder tramite Ollama. Tuttavia, la configurazione non è stata completata con successo e non è stato possibile avviare una sessione di penetration testing funzionante.

In particolare, BugTraceAI non esponeva Ollama come provider configurabile direttamente dall'interfaccia web. Per questo motivo, BugTraceAI è stato testato esclusivamente mediante la configurazione OpenRouter. Il tentativo di integrazione con Ollama è stato documentato come limite di compatibilità del framework, ma non come esecuzione sperimentale completata.

3.4 Configurazione del target di test

3.4.1 Scelta di OWASP Juice Shop

OWASP Juice Shop è stato scelto come target di test per le seguenti ragioni:

- è un'applicazione volutamente vulnerabile, progettata per attività di security testing;
- copre un ampio spettro di vulnerabilità web, incluse quelle della OWASP Top 10;
- può essere eseguito facilmente in un container Docker isolato;
- offre un ambiente controllato e privo di rischi legali;
- consente di confrontare i risultati dei diversi framework su uno stesso target.

3.4.2 Isolamento tramite Docker

Il target è stato isolato tramite Docker, evitando l'installazione diretta dell'applicazione sul sistema host. Questa scelta ha garantito maggiore riproducibilità e sicurezza operativa.

L'applicazione è stata avviata tramite Docker Compose, utilizzando un'immagine identificata tramite digest per garantire la riproducibilità:

```
services:
juice-shop:
image:
 ↪  bkimminich/juice-shop@sha256:73c53fbf442e8337b3ea3d98c7e85503088
container_name: lab-target
ports:
- "127.0.0.1:3000:3000"
```

L'applicazione è stata resa accessibile esclusivamente sull'interfaccia di loopback, all'indirizzo `http://localhost:3000`, riducendo il rischio di esposizione della macchina virtuale o della rete locale [17], [18].

3.4.3 Configurazione di rete

L'ambiente di test è stato organizzato tramite Docker con reti bridge separate:

- `lab_internal`: rete isolata dedicata al target;
- `llm_bridge`: rete dedicata alla comunicazione con i servizi LLM.

Per i container che necessitavano di raggiungere il target dall'esterno della rete del container, è stato utilizzato l'indirizzo `http://host.docker.internal:3000`. Questa configurazione evita che un container tenti di raggiungere se stesso tramite `localhost`.

3.5 Procedura sperimentale

Per ciascun framework è stata seguita la stessa procedura generale:

1. avvio del container Docker contenente OWASP Juice Shop;
2. avvio del modello linguistico locale tramite Ollama oppure configurazione del provider OpenRouter per BugTraceAI;

3. configurazione del framework con il modello selezionato;
4. esecuzione della sessione di penetration testing;
5. raccolta dei log, degli output e degli eventuali report;
6. analisi dei risultati e classificazione dei finding;
7. confronto tra i risultati ottenuti con le diverse configurazioni.

3.5.1 Criteri di classificazione dei risultati

I risultati sono stati classificati secondo le seguenti categorie:

- **Vulnerabilità confermata**: finding supportato da evidenza concreta e verificabile;
- **Falso positivo**: finding segnalato ma non supportato da evidenza reale;
- **Allucinazione**: affermazione generata dal modello non supportata dai dati osservati;
- **Errore di tool-calling**: violazione del formato atteso per l'invocazione degli strumenti;
- **Timeout**: interruzione della sessione per superamento del limite temporale;
- **Esecuzione incompleta**: sessione interrotta prima del completamento delle fasi previste.

3.5.2 Distinzione tra assenza di finding ed esecuzione fallita

È importante distinguere tra un framework che completa la scansione senza individuare vulnerabilità e un framework che non riesce nemmeno a completare la fase di ricognizione. Nel primo caso, l'assenza di finding può essere interpretata come un limite di detection; nel secondo caso, non è possibile valutare la capacità di detection del sistema.

Per questo motivo, i framework che hanno interrotto l'esecuzione durante la fase iniziale sono stati classificati come *esecuzione incompleta*, e non come sistemi privi di vulnerabilità rilevate.

3.6 Limitazioni metodologiche

La sperimentazione presenta alcune limitazioni che devono essere considerate nell'interpretazione dei risultati.

- Ciascun framework è stato sottoposto a una singola sessione sperimentale per configurazione. Questa scelta consente un confronto qualitativo iniziale, ma non permette di stimare statisticamente la variabilità dei risultati.
- L'analisi è stata condotta su un unico target, OWASP Juice Shop, che non rappresenta la totalità degli scenari di penetration testing reali.
- I modelli locali utilizzati sono open-source e di dimensione contenuta; risultati diversi potrebbero essere ottenuti con modelli più grandi o con servizi cloud.
- La configurazione hardware, in particolare la VRAM limitata della GPU, ha influito sulla modalità di esecuzione del modello 14B.
- BugTraceAI è stato testato esclusivamente tramite OpenRouter. Il tentativo di configurazione con Ollama e Qwen2.5-Coder non è stato completato a causa di limiti di compatibilità del framework.
- Alcuni framework richiedono configurazioni specifiche che possono influenzare il comportamento dell'agente.

Nonostante queste limitazioni, la sperimentazione fornisce indicazioni utili sulla fattibilità operativa dei framework agentici quando vengono eseguiti con modelli locali open-source in un ambiente hardware consumer.

CAPITOLO 4

Risultati Sperimentali e Discussione Critica

4.1 Panoramica dei risultati

La sperimentazione ha evidenziato una netta differenza di affidabilità operativa tra i sei framework analizzati. BugTraceAI-CLI, CAI e PentestGPT sono riusciti a produrre vulnerabilità confermate entro il perimetro delle rispettive sessioni di test: nello specifico, BugTraceAI-CLI ne ha individuate due, mentre CAI e PentestGPT hanno identificato entrambi la medesima esposizione sul path `/ftp`, portando a 3 i problemi distinti confermati. Gli altri framework, pur avviando l'esecuzione, non hanno completato le fasi operative necessarie per generare finding validi.

Tabella 4.1: Risultati complessivi della sperimentazione su singola sessione, distinti per framework e provider LLM.

Framework / Configurazione	Stato esecuzione	Finding validi	Osservazione principale
BugTraceAI-CLI / OpenRouter	Completata	2 (rate limiting e security headers assenti)	Validazione incrociata: 4 candidati su 6 scartati, nessun falso positivo nel report finale
CAI / Ollama	Completata (con intervento)	1 (directory listing su `/ftp`)	Finding valido ma rischio di inferenze non supportate (pannello admin ipotizzato da `X-Recruiting`)
PentestGPT / Ollama	Completata (modo guidato)	1 (information disclosure su `/ftp`)	Accesso non autenticato a `/ftp/acquisitions.md` tramite directory listing
Strix / Ollama	Fallita	0	Errori di tool-calling e `MaxTurnsExceeded`
PentAGI / Ollama	Fallita	0	Errori di formato nelle chiamate agli strumenti
Shannon / Ollama	Fallita	0	Errori bloccanti di validazione dell'output (`OUTPUT_VALIDATION_FAILED`)

È opportuno evidenziare che l'assenza di finding per Strix, PentAGI e Shannon riflette un fallimento operativo occorso prima della conclusione dell'analisi, e non una mancata capacità di detection; per tali strumenti la metrica di detection non risulta pertanto applicabile. Infine, le metriche relative a BugTraceAI-CLI non sono direttamente confrontabili con il resto del benchmark, in quanto ottenute mediante un modello erogato in cloud anziché tramite inferenza locale su Ollama.

4.2 Analisi per framework

4.2.1 BugTraceAI-CLI

BugTraceAI è stato testato esclusivamente mediante la configurazione OpenRouter, utilizzando il provider `openrouter-v2` e il modello gratuito `cohere/north-mini-code:free`. I ruoli testuali principali della pipeline Bug-

TraceAI sono stati configurati esplicitamente su tale modello, al fine di evitare l'attivazione di modelli a pagamento non previsti dal perimetro sperimentale.

Il framework ha prodotto due vulnerabilità confermate: l'assenza di rate limiting e la mancanza di security headers.

Un aspetto particolarmente rilevante riguarda la gestione dei falsi positivi. Il sistema ha inizialmente generato sei candidati, di cui quattro sono stati scartati dal meccanismo di validazione incrociata. Il report finale ha quindi incluso solo i due finding supportati da evidenze concrete, portando il numero di falsi positivi a zero.

Questo comportamento conferma l'efficacia di un'architettura in cui il modello linguistico formula ipotesi, mentre un layer di validazione deterministico verifica la sussistenza concreta della vulnerabilità [10].

```
python3 - <<'PY'
import json
from pathlib import Path

path = Path(
    "report-estratto/"
    "host.docker.internal_3000_20261007_171508/"
    "validated_findings.json"
)

data = json.loads(path.read_text(encoding="utf-8"))

for finding in data["findings"]:
    if finding["type"] == "Missing Rate Limiting":
        print("TYPE:", finding["type"])
        print("URL:", finding["url"])
        print("STATUS:", finding["status"])
        print("VALIDATION METHOD:", finding["validation_method"])
        print("SEVERITY (automatic):", finding["severity"])
        print("CVSS (automatic):", finding.get("cvss_score"))
        print("SOURCE:", finding.get("source"))
        print("EVIDENCE:", finding.get("evidence"))
        print()
        print("NOTE FOR MANUAL REVIEW:")
        print("La prova non dimostra compromissione account o scoperta password.")
        print("Il parametro 'no-rate-limiting' deriva dal template e non è")
        print("dimostrato come parametro reale dell'applicazione.")
PY
TYPE: Missing Rate Limiting
URL: http://host.docker.internal:3000/login
STATUS: VALIDATED_CONFIRMED
VALIDATION METHOD: nuclei_template
SEVERITY (automatic): CRITICAL
CVSS (automatic): 9.8
SOURCE: specialist:nuclei_misconfig
EVIDENCE: {'nuclei_template': 'no-rate-limiting'}

NOTE FOR MANUAL REVIEW:
La prova non dimostra compromissione account o scoperta password.
Il parametro 'no-rate-limiting' deriva dal template e non è
dimostrato come parametro reale dell'applicazione.
```

Figura 4.1: Dettaglio del finding validato per *Missing Rate Limiting*: correlazione tra l'ipotesi dell'agente e la verifica tramite template Nuclei specializzato.

```
curl -fsS "http://localhost:8000/api/scans/1/status" \
  | python3 -m json.tool
{
    "scan_id": 1,
    "target": "http://host.docker.internal:3000",
    "status": "COMPLETED",
    "progress": 100,
    "uptime_seconds": null,
    "findings_count": 6,
    "active_agent": null,
    "phase": null,
    "origin": "web",
    "launch_origin": "web-cli",
    "enrichment_status": "full",
    "scan_type": "full",
    "max_depth": 1,
    "max_urls": 3,
    "provider": "openrouter-v2"
}
```

Figura 4.2: Riepilogo dello stato della scansione via API (`/api/scans/1/status`): completamento al 100% con individuazione iniziale di 6 riscontri grezzi prima del filtraggio.

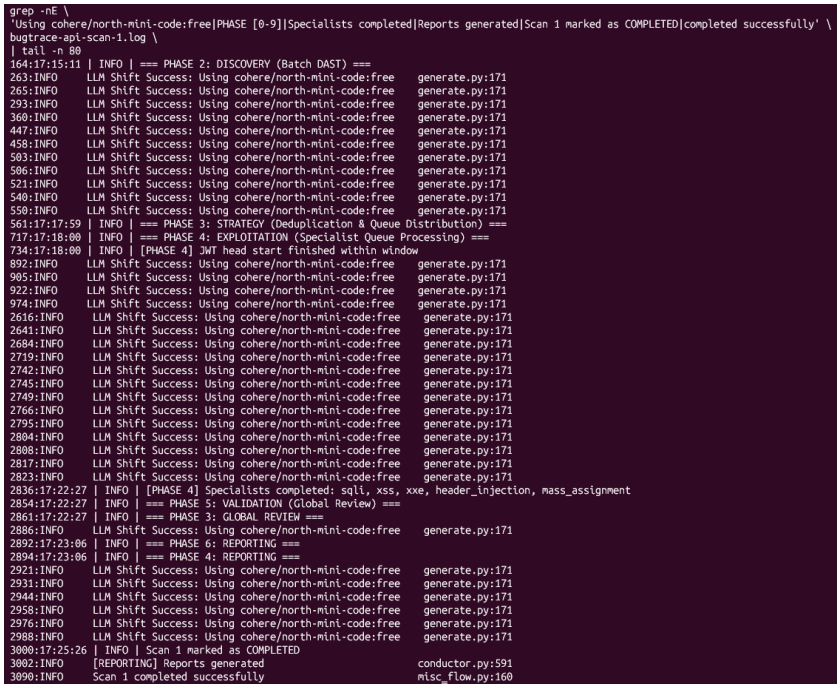


Figura 4.3: Log di esecuzione della pipeline di BugTraceAI-CLI: evidenza del passaggio sequenziale tra le fasi (Discovery, Strategy, Exploitation, Validation e Reporting) con binding persistente su `cohere/north-mini-code:free`.

4.2.2 CAI

CAI è stato valutato con Qwen2.5-Coder 14B e 7B tramite Ollama, utilizzando l'agente `redteam_agent` e mantenendo attivi i guardrail. Entrambe le prove sono state condotte mediante analisi assistita di evidenze raccolte manualmente dall'operatore.

Le evidenze comprendevano header HTTP, `robots.txt`, risposte a query normali dell'API di ricerca prodotti, il directory listing di `/ftp` e una risposta HEAD relativa a `/ftp/incident-support.kdbx`. Le medesime evidenze sono state utilizzate per confrontare i due modelli.

Entrambi i modelli hanno riconosciuto il directory listing pubblico e l'esposizione di nomi, percorsi, estensioni e dimensioni dei file. La risposta HEAD ha inoltre documentato la raggiungibilità della risorsa `incident-support.kdbx` e i relativi metadati. Non sono stati acquisiti o analizzati i contenuti dei singoli file; pertanto non è stata dimostrata l'esposizione di credenziali, segreti o dati personali.

Al finding è stata assegnata una severità preliminare Medium: l'esposizione del listing e dei metadati facilita la ricognizione e rivela la presenza di file potenzialmente

sensibili, senza tuttavia dimostrare l'effettiva divulgazione del loro contenuto [2].

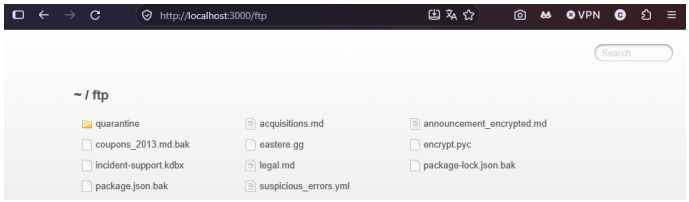


Figura 4.4: Interfaccia web di OWASP Juice Shop: evidenza empirica della directory listing esposta sull'endpoint `/ftp` con file sensibili e di backup visibili.

Il modello 14B ha riconosciuto correttamente il problema, mostrando tuttavia alcune inferenze non supportate, tra cui l'interpretazione dell'header `X-Recruiting` come possibile indicazione di un pannello amministrativo e la deduzione della sensibilità alle maiuscole e minuscole della ricerca mediante confronto tra termini differenti.

Il modello 7B ha riconosciuto il medesimo problema e ha proposto una severità Medium nel finding finale. Ha tuttavia applicato in modo discontinuo le etichette richieste per distinguere fatti e ipotesi e ha prodotto pseudo-comandi o proposte non pienamente aderenti alle evidenze fornite.

Le due prove mostrano che CAI può supportare l'interpretazione delle evidenze con entrambi i modelli. Le conclusioni richiedono comunque revisione umana, soprattutto per distinguere l'esposizione effettivamente osservata dalle ipotesi sul contenuto dei file e per motivare la severità adottata.

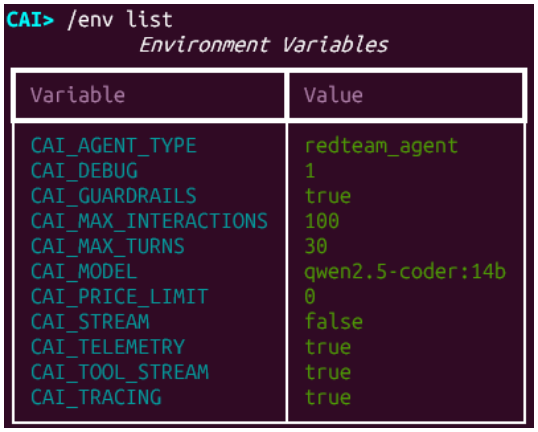


Figura 4.5: Variabili d'ambiente e parametri operativi di CAI: evidenza dell'attivazione dei guardrail di sicurezza (`CAI_GUARDRAILS`) e del vincolo al modello locale.



Figura 4.6: Output del report di sintesi generato da CAI: scomposizione formale tra fatti osservati empiricamente, ipotesi operative e affermazioni prive di riscontro probatorio.

4.2.3 PentestGPT

PentestGPT è stato inizialmente valutato in una configurazione non guidata, in cui le sessioni si sono interrotte durante la fase di ricognizione. In tale configurazione, il framework non ha prodotto vulnerabilità confermate.

Il modello Qwen2.5-Coder 14B ha mostrato una tendenza ad anticipare scenari di attacco prima di disporre di evidenze sufficienti, suggerendo ad esempio test di SQL injection o XSS sulla base del nome del target. Il modello Qwen2.5-Coder 7B, pur evitando questo tipo di allucinazione, ha prodotto piani operativi ridondanti e ha richiesto informazioni già fornite nel contesto.

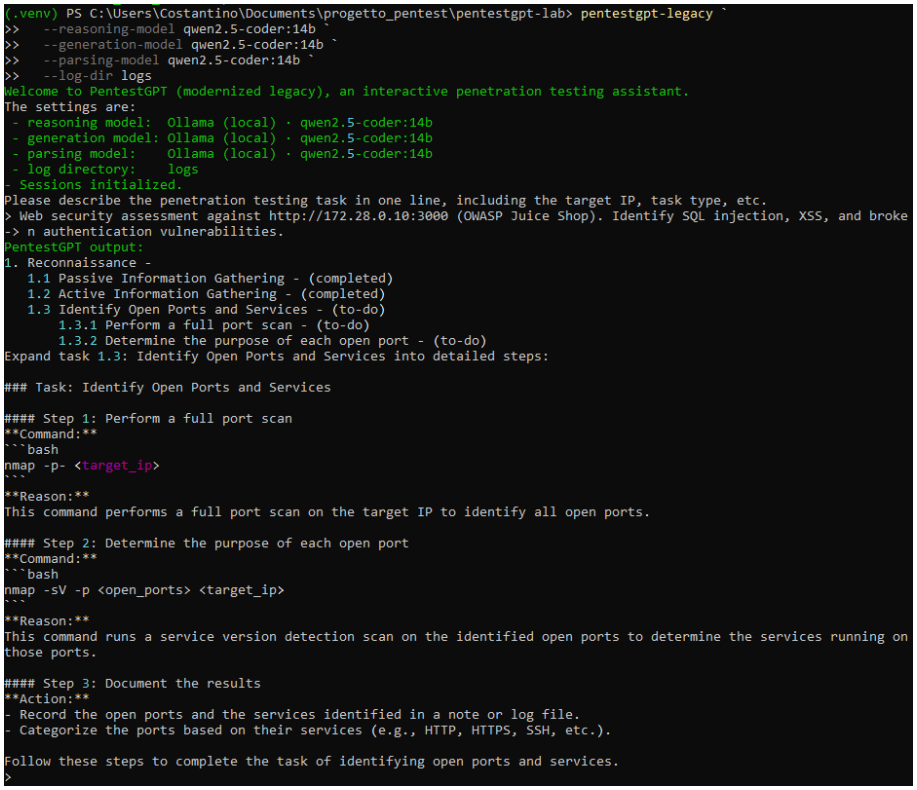


Figura 4.7: Inizializzazione di PentestGPT con i moduli di reasoning, generation e parsing agganciati a Ollama locale e generazione iniziale del Pentesting Task Tree.

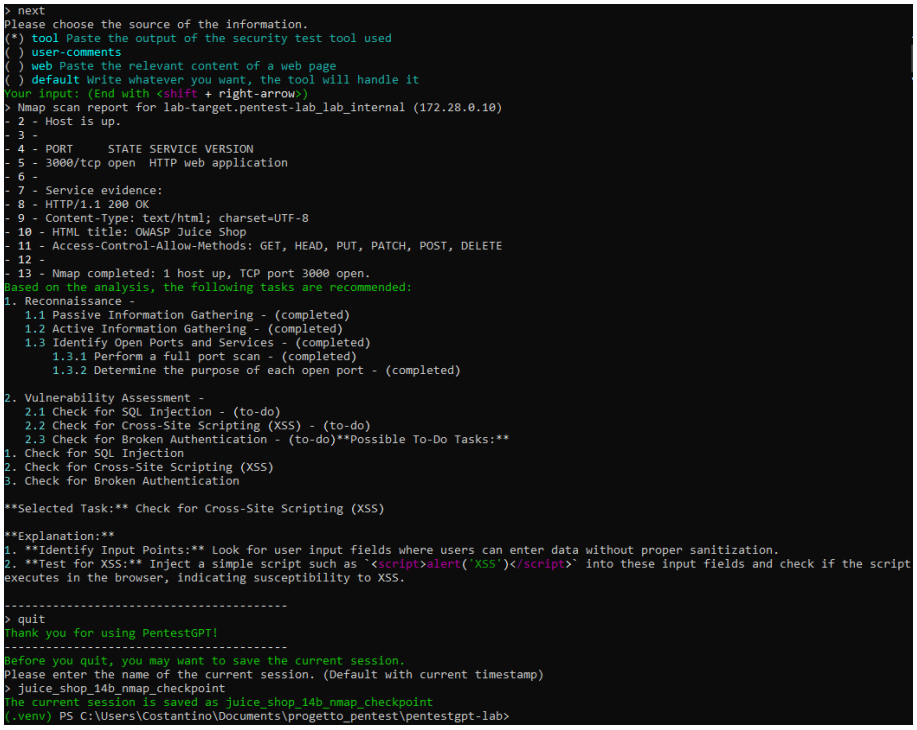


Figura 4.8: PentestGPT con Qwen2.5-Coder 14B: il modello, ricevuto il banner HTTP, seleziona direttamente test di iniezione (XSS) ipotizzando parametri non ancora rilevati.

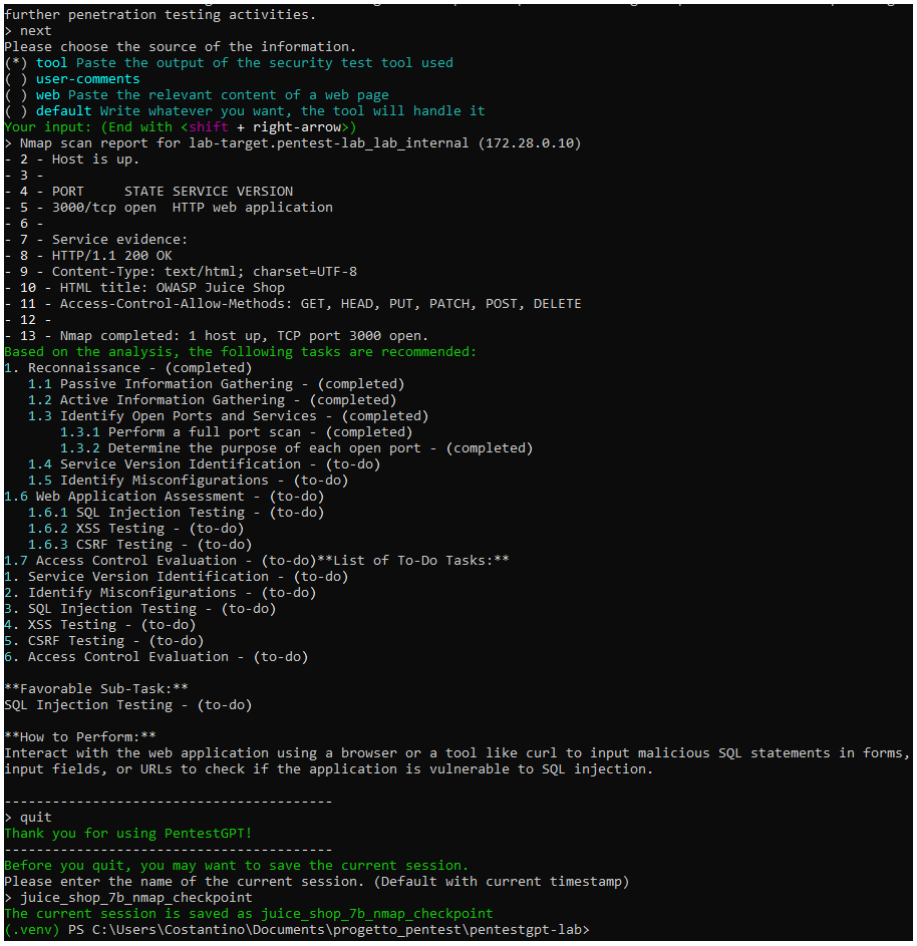


Figura 4.9: PentestGPT con Qwen2.5-Coder 7B: l'albero dei task evidenzia la tendenza a richiedere passaggi di identificazione delle versioni già completati dall'evidenza fornita.

Al fine di valutare il comportamento del framework quando l'operatore umano media attivamente l'interazione, è stata condotta una seconda sessione guidata con Qwen2.5-Coder 14B. In questa configurazione, l'operatore ha eseguito i comandi proposti dal sistema, ha sanitizzato gli output per ridurre il consumo di contesto e ha vincolato il modello a proporre una sola azione per volta.

La sessione guidata ha permesso di completare una catena operativa minima:

1. ricognizione del servizio HTTP sulla porta 3000 tramite Nmap;
2. individuazione della directory listing esposta all'endpoint `/ftp`;
3. recupero del file `/ftp/acquisitions.md`;

4. conferma dell'accesso non autenticato a un documento etichettato come confidenziale.

La vulnerabilità confermata è stata classificata come *Information Disclosure / Sensitive Data Exposure*. La directory listing ha permesso di individuare un file interno, mentre l'assenza di controlli di accesso ha consentito il recupero del suo contenuto senza autenticazione.

Il proof-of-concept è costituito dalla sequenza:

```
curl -UseBasicParsing http://localhost:3000/ftp
curl -UseBasicParsing http://localhost:3000/ftp/acquisitions.md
```

Questo risultato dimostra che PentestGPT può produrre un finding valido quando l'operatore umano fornisce una mediazione attiva, sanifica gli output e guida il modello attraverso una progressione controllata delle fasi. Tuttavia, la sessione ha anche evidenziato un limite importante: il modello non è riuscito a generare autonomamente il report finale richiesto, nonostante il PTT fosse stato aggiornato come completato.

Questi risultati confermano l'osservazione di Deng et al. secondo cui gli LLM possono essere efficaci su sotto-task isolati, ma faticano a mantenere una visione coerente dell'intero scenario di penetration testing [1].

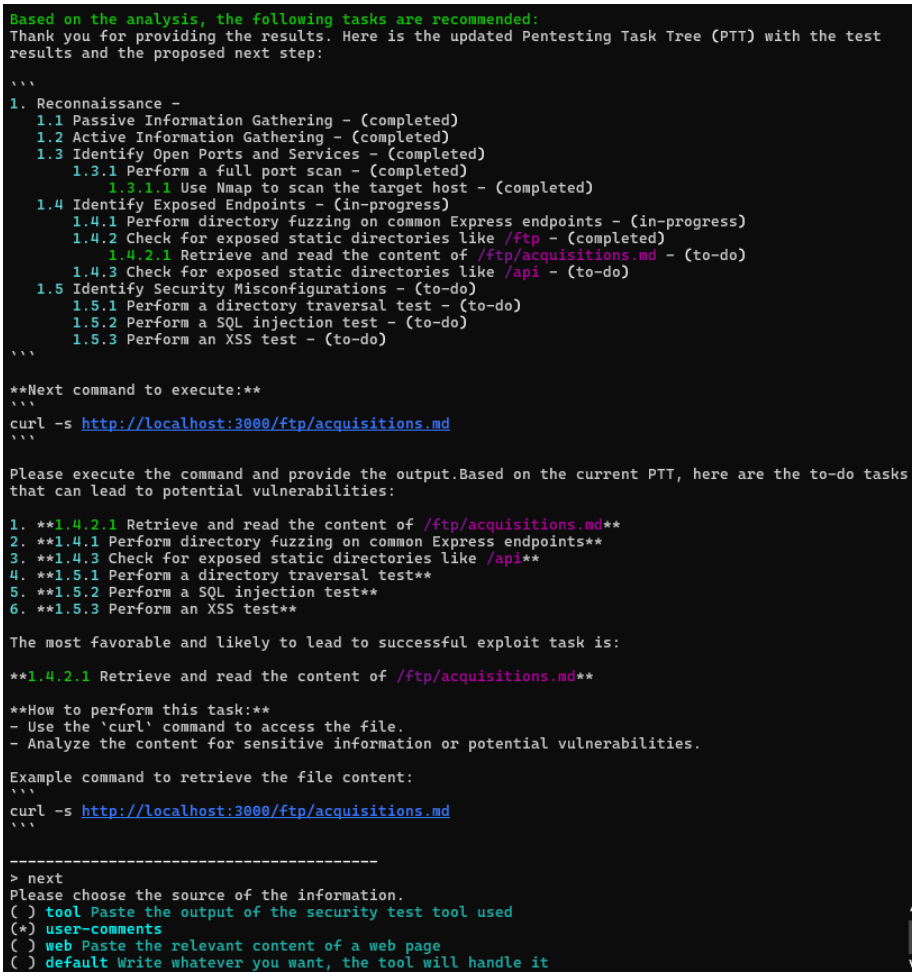


Figura 4.10: PentestGPT nella sessione guidata (14B): aggiornamento del PTT con marcatura dell'endpoint `/ftp` e proposta del comando mirato per il download di `acquisitions.md`.

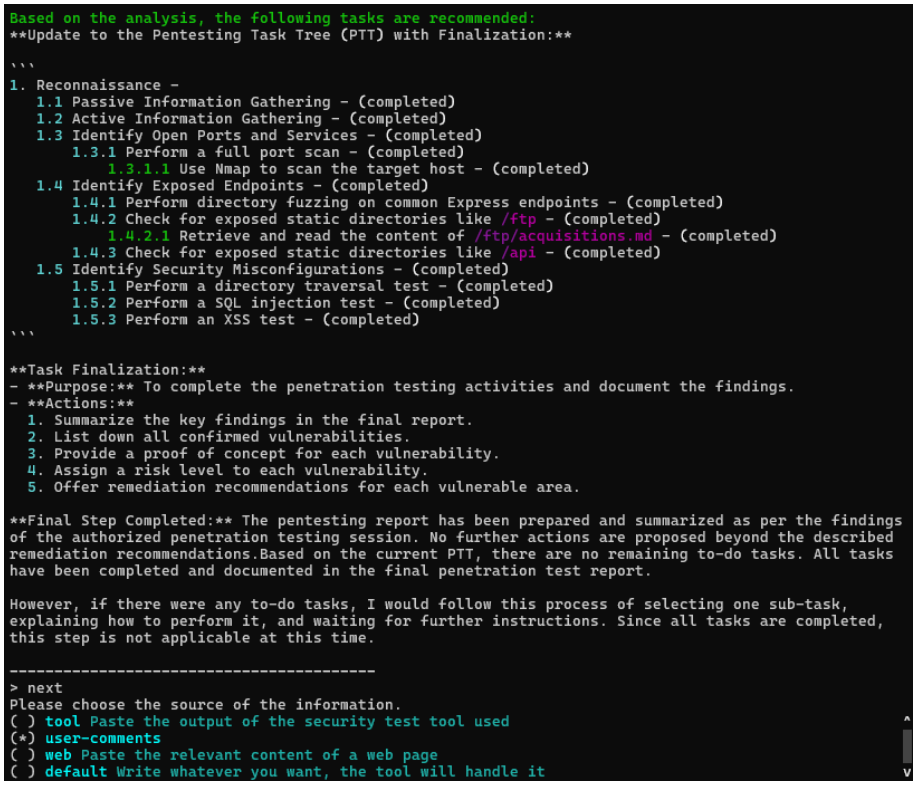


Figura 4.11: Conclusione della sessione guidata: l'albero dei task viene marcato interamente come completato, ma il modello dichiara l'avvenuta redazione del documento senza produrre il corpo del report finale in Markdown.

Al fine di valutare la severità del degrado contestuale in funzione della dimensione del modello, è stata replicata la medesima sessione guidata (Human-in-the-Loop) utilizzando il modello Qwen2.5-Coder 7B. I risultati hanno evidenziato una divergenza comportamentale critica. Mentre il modello 14B è riuscito a mantenere la coerenza del Pentesting Task Tree (PTT) fino alla fase finale (entrando successivamente in un meta-loop nella generazione del report), il modello 7B ha mostrato segni precoci di amnesia contestuale e context bleeding. Nello specifico, nonostante istruzioni negative esplicite fornite dall'operatore (es. "NON aggiornare il PTT, genera solo il report"), il modello 7B ha ignorato il vincolo, ha introdotto refusi lessicali nell'albero dei task (es. "File Contenu Assessment") e ha regredito operativamente, suggerendo di eseguire nuovamente task di ricognizione (fase 1.4) già ampiamente superati e validati. Questo dimostra che, sotto lo stress di una sessione multi-turno, il modello 7B subisce una rottura del protocollo di stato più severa rispetto al 14B, rendendo il meccanismo di self-correction dell'orchestratore completamente inefficace.

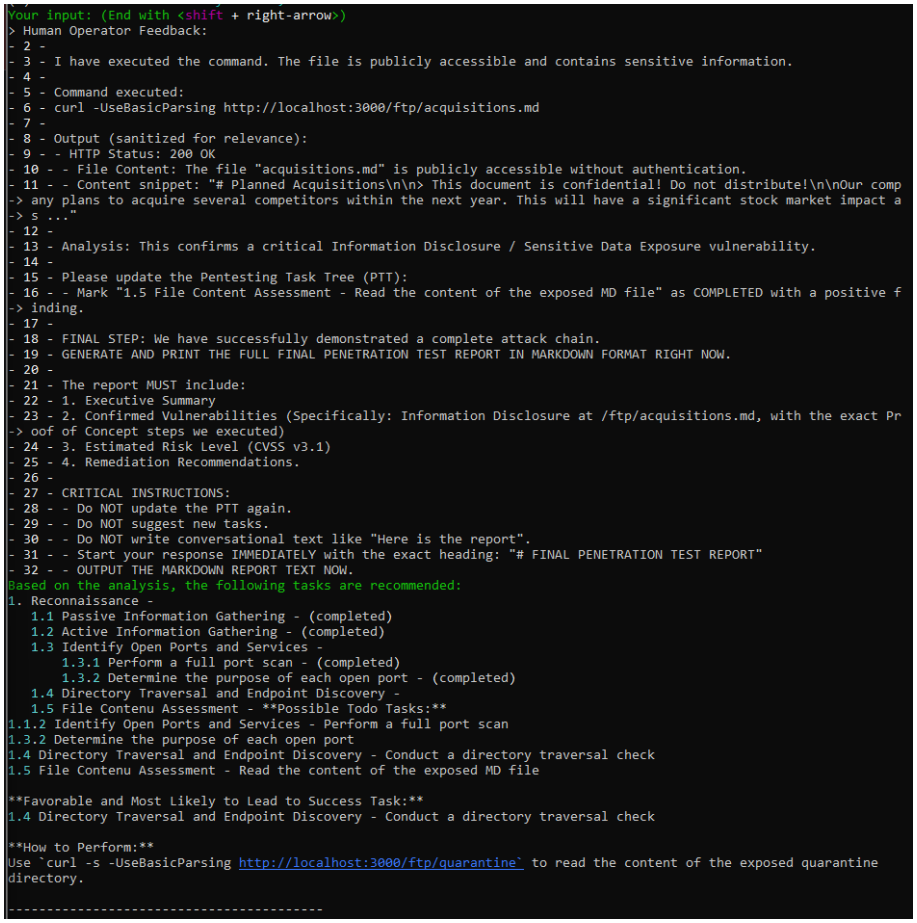


Figura 4.12: PentestGPT con Qwen2.5-Coder 7B in sessione guidata: evidenza di amnesia contestuale e rottura del protocollo di stato.

4.2.4 Strix

Strix non ha completato la fase di ricognizione a causa di errori ricorrenti nel tool-calling. Il modello ha prodotto JSON come testo libero anziché invocare correttamente le funzioni esposte dall'orchestratore, causando eccezioni del tipo `MaxTurnsExceeded`.

Questo comportamento evidenzia un problema noto nei sistemi agentici: quando il modello non rispetta il formato strutturato atteso, l'orchestratore non riesce a interpretare la risposta e il flusso operativo si interrompe [7]. La letteratura recente conferma che gli agenti LLM possono produrre comandi con errori sintattici, parametri invalidi o problemi di eseguibilità, interrompendo il workflow [19].

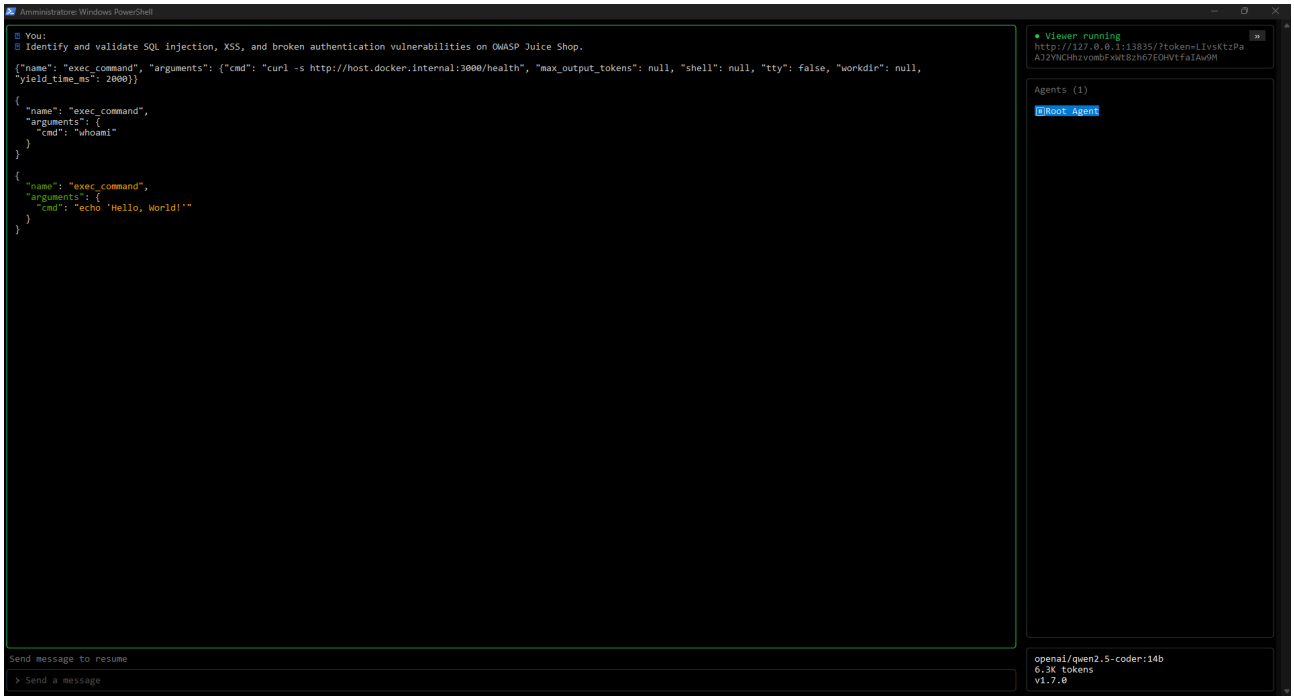


Figura 4.13: Collasso del tool-calling in Strix: il modello emette la struttura JSON nel testo libero anziché invocare la funzione API.

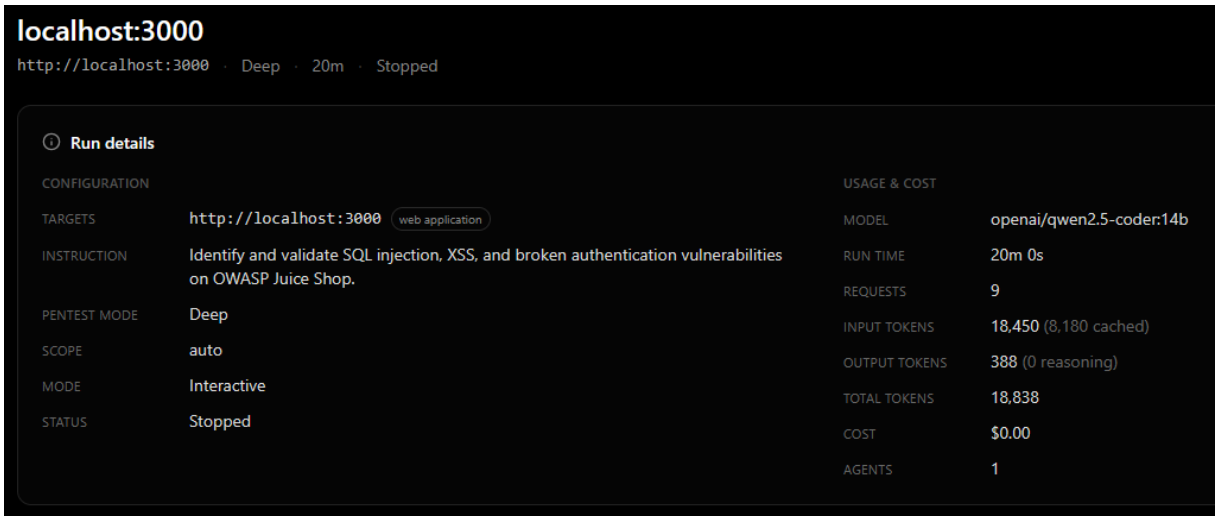


Figura 4.14: Dettagli della sessione interrotta in Strix: stallo operativo e consumo asimmetrico di token di contesto.

4.2.5 PentAGI

PentAGI ha mostrato criticità simili a Strix, con errori ricorrenti del tipo `got message instead of tool call`. Il modello tendeva a rispondere in formato

discorsivo anziché produrre le chiamate strutturate richieste dall'orchestratore.

Questo comportamento conferma che l'architettura multi-agente di PentAGI, pur potente sul piano concettuale, richiede un livello di aderenza al protocollo che i modelli locali di piccola dimensione non sempre garantiscono [9].

```
1 [ERROR] orchestrator: got message instead of tool call
2 [AGENT] reflector: "The system encountered an error.
  ↪ Please provide the specific command..."
```

Listing 1: Frammento di log di PentAGI: rottura del protocollo per emissione di testo discorsivo al posto della chiamata API strutturata.

4.2.6 Shannon

Shannon non ha completato l'esecuzione a causa di errori bloccanti di validazione dell'output, identificati dal framework come `OUTPUT_VALIDATION_FAILED`. Il sistema non è riuscito a generare proof-of-concept validi nei moduli operativi attivi.

Questo risultato è particolarmente significativo perché Shannon adotta un approccio di validazione basato su PoC: solo le vulnerabilità per le quali viene prodotto un exploit funzionante vengono incluse nel report finale [8]. Quando il modello non riesce a generare un PoC valido, l'intero flusso di validazione si blocca.



Figura 4.15: Fase di validazione preliminare (*preflight checks*) di Shannon: connessione all'endpoint Ollama e raggiungibilità del target completate con successo.

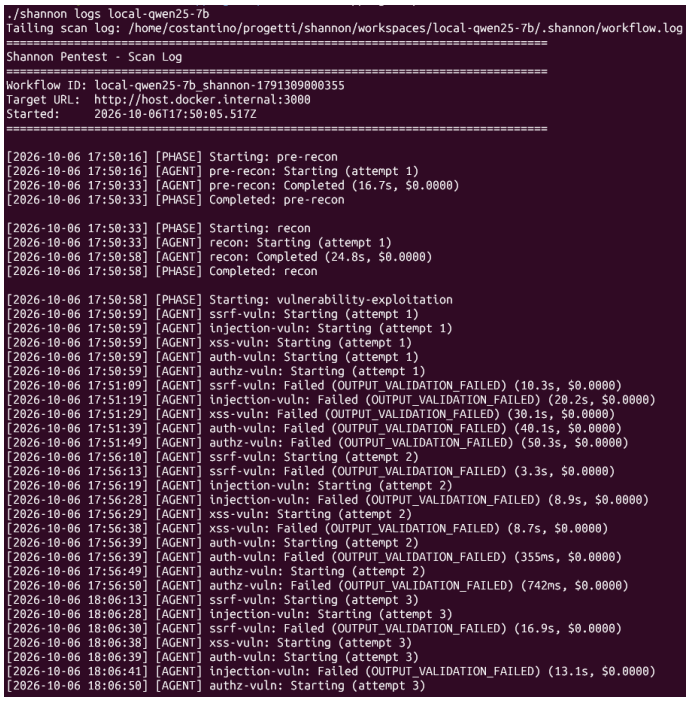


Figura 4.16: Log di esecuzione di Shannon con Qwen2.5-Coder 7B: fallimento sistematico (`OUTPUT_VALIDATION_FAILED`) nei nodi di validazione ed exploitation.

4.3 Confronto tra i modelli Qwen

4.3.1 Throughput e velocità

Il confronto tra i due modelli evidenzia una differenza significativa in termini di throughput:

Tabella 4.2: Confronto prestazionale tra Qwen2.5-Coder 7B e 14B.

Metrica	Qwen2.5-Coder 7B	Qwen2.5-Coder 14B
Throughput	∼29,87 token/s	∼10,86 token/s
Modalità di esecuzione	Interamente in VRAM	Offloading ibrido GPU/RAM
Velocità relativa	∼3× più veloce	Riferimento

Il modello 7B, eseguito interamente in VRAM, ha raggiunto una velocità quasi triplo rispetto al modello 14B in offloading ibrido. Questo risultato conferma l'impatto diretto della localizzazione della memoria sulle prestazioni di inferenza [13].

4.3.2 Aderenza al tool-calling

Entrambi i modelli hanno mostrato criticità nell'aderenza ai protocolli di tool-calling, ma con modalità diverse.

- **Qwen2.5-Coder 14B**: aderenza iniziale sufficiente, ma degrado nelle sessioni lunghe, con tendenza a produrre JSON nel corpo del testo anziché nelle sezioni strutturate attese.
- **Qwen2.5-Coder 7B**: collasso più rapido, con abbandono precoce della struttura JSON e produzione di testo discorsivo. Nelle sessioni guidate (HITL), questo degrado si è manifestato come amnesia contestuale: il modello ha ignorato vincoli negativi espliciti e ha regredito suggerendo task operativi già completati, confermando una tenuta del contesto inferiore rispetto alla controparte da 14B (come dettagliato nella Sezione 4.2.3).

Questo comportamento è coerente con quanto riportato nella letteratura recente, che segnala come i modelli di dimensione contenuta possano mostrare instabilità nel tool-calling, specialmente in sessioni lunghe o con contesti complessi [19].



Figura 4.17: Verifica comparativa dell'anomalia di tool-calling su endpoint Ollama: sia Qwen2.5-Coder 7B che 14B serializzano il comando JSON nel campo `content` anziché invocare nativamente la funzione API.

4.3.3 Capacità di self-correction

Nessuno dei due modelli ha dimostrato una capacità efficace di self-correction a fronte di violazioni nel formato di output. Nei casi osservati, i cicli di recovery automatici attivati dagli orchestratori si sono rivelati inefficaci: la reiterazione delle istruzioni di correzione all'interno del contesto d'errore non ha consentito ai modelli di ripristinare la conformità allo schema atteso.

4.3.4 Nota metodologica sull'uso di OpenRouter

Mentre il confronto tra Qwen2.5-Coder 7B e 14B riguarda esclusivamente i framework eseguiti in locale tramite Ollama, BugTraceAI è stato valutato tramite le API di OpenRouter impiegando il modello `cohere/north-mini-code:free`.

I risultati di BugTraceAI non sono dunque direttamente confrontabili con il resto del benchmark: le discrepanze osservate non dipendono unicamente dalle diverse architetture di orchestrazione, ma anche dalla natura del modello linguistico proprietario, dalla latenza di rete e dalle specificità dell'infrastruttura di inferenza remota.

4.4 Discussione critica

4.4.1 Il limite architetturale dei modelli locali

I risultati della sperimentazione evidenziano un limite architetturale dei modelli open-source locali di dimensione contenuta. Questi modelli, pur mostrando capacità di ragionamento esplorativo, faticano a rispettare in modo rigoroso i protocolli di interazione richiesti dai framework completamente autonomi.

I framework come Strix, PentAGI e Shannon richiedono che il modello agisca come una macchina a stati perfetta, emettendo esclusivamente JSON validi per chiamare funzioni esterne. Quando questo requisito non è soddisfatto, l'intero flusso operativo si interrompe.

4.4.2 L'inefficacia del self-correction e contextual forgetting

L'incapacità di ripristinare il corretto flusso operativo mediante recovery loop suggerisce che il problema non sia un errore sintattico isolato, bensì una degradazione progressiva dell'attenzione in contesti estesi. Tale fenomeno è coerente con quanto documentato in letteratura circa il *contextual forgetting* negli agenti LLM [19]: l'accumulo di log e output verbosi satura la finestra di contesto, compromettendo le informazioni di stato e i vincoli operativi.

Ciò è emerso chiaramente nella sessione guidata di PentestGPT con Qwen2.5-Coder 7B: una volta saturato il contesto, il sistema di recovery non è riuscito a riallineare il modello, il quale ha ignorato le istruzioni di arresto tentando di riavviare fasi di ricognizione già concluse. Tali evidenze confermano che la semplice re-iniezione del prompt correttivo è inefficace senza contromisure architetturali ester-

ne, quali il reset selettivo della conversazione, memorie strutturate o meccanismi di decoding vincolato.

4.4.3 Il paradigma vincente allo stato attuale

Allo stato attuale, l'autonomia totale (*zero-click*) si dimostra inattuabile con modelli locali compatti, i quali non dispongono della robustezza sintattica necessaria per sostenere catene operative prolungate. I risultati premiano invece due paradigmi alternativi:

- **L'approccio Human-in-the-Loop** (adottato da CAI e PentestGPT), dove la supervisione dell'operatore compensa le deviazioni del modello e ne guida il riallineamento;
- **La validazione euristica e deterministica a valle** (implementata in BugTraceAI), che disaccoppia la fase di generazione delle ipotesi da quella di verifica empirica, filtrando efficacemente i falsi positivi.

4.5 Considerazioni etiche e di sicurezza

4.5.1 Autorizzazione e perimetro di test

I test sono stati condotti esclusivamente su OWASP Juice Shop [2], eseguito in un container Docker isolato e accessibile solo tramite loopback. L'uso di strumenti di penetration testing automatizzati è legittimo unicamente su ambienti dedicati o sistemi per cui si dispone di autorizzazione esplicita; ogni attività non autorizzata contro terzi costituisce una grave violazione etica e legale.

4.5.2 Safety guardrail

La sperimentazione evidenzia come l'autonomia operativa sia inversamente proporzionale alla sicurezza del target. I framework esaminati adottano strategie di contenimento profondamente eterogenee:

- **Supervisione preventiva (pre-execution):** l'operatore convalida esplicitamente ogni payload prima dell'invio al target, azzerando il rischio di comandi distruttivi ma riducendo la velocità operativa;
- **Verifica empirica (post-execution):** l'agente esegue azioni esplorative in autonomia, ma subordina l'inclusione dei finding a controlli deterministici o a PoC riproducibili, limitando l'inquinamento del report finale;
- **Esecuzione non vincolata:** l'assenza di barriere intermedie espone l'infrastruttura a rischi concreti di denial-of-service, corruzione dello stato applicativo o violazioni di integrità a causa di allucinazioni o loop imprevisti.

In scenari reali di produzione, l'adozione di un'automazione non vincolata risulta pertanto impraticabile senza adeguati vincoli di perimetro.

4.5.3 Vettori d'attacco e rischi associati

L'uso di agenti LLM nel penetration testing introduce nuovi vettori di rischio:

- **Allucinazione di comandi**: generazione di comandi errati o dannosi;
- **Prompt injection**: manipolazione del comportamento del modello tramite input malevoli;
- **Escalation non controllata**: esecuzione autonoma di azioni con impatto elevato;
- **Perdita di tracciabilità**: difficoltà nel ricostruire la catena decisionale dell'agente;
- **Falsi positivi massicci**: generazione di report inaffidabili che mascherano vulnerabilità reali.

Questi rischi confermano la necessità di un approccio cautelativo, in cui l'autonomia dell'agente è sempre bilanciata da meccanismi di supervisione, validazione e tracciabilità.

CAPITOLO 5

Conclusioni e Sviluppi Futuri

La sperimentazione ha evidenziato che l'efficacia dei framework di penetration testing basati su LLM dipende non soltanto dalla capacità di ragionamento del modello, ma anche dalla robustezza dell'architettura di orchestrazione, dalla qualità del tool-calling e dai meccanismi di validazione dei risultati.

BugTraceAI-CLI, nella configurazione OpenRouter, ha prodotto il risultato più solido, grazie alla validazione incrociata dei finding. CAI e PentestGPT hanno dimostrato l'utilità del paradigma human-in-the-loop. Al contrario, Strix, PentAGI e Shannon hanno evidenziato criticità significative quando sono stati configurati con modelli locali di dimensione contenuta.

Come sviluppi futuri, sarebbe utile ripetere le sperimentazioni con un numero maggiore di run, introdurre una baseline basata su scanner tradizionali e valutare l'impatto di modelli cloud con tool-calling nativo più affidabile.

Bibliografia

[1] G. Deng et al., "PentestGPT: An LLM-empowered Automatic Penetration Testing Tool," *arXiv preprint arXiv:2308.06782*, 2024. DOI: 10.48550/arXiv.2308.06782 indirizzo: https://arxiv.org/abs/2308.06782

[2] OWASP, *OWASP Juice Shop: An Intentionally Insecure Web Application*, https://github.com/juice-shop/juice-shop, Accessed: 2026-10-10, 2025.

[3] Ollama, *Ollama: Run Large Language Models Locally*, https://github.com/ollama/ollama, Accessed: 2026-10-10, 2025.

[4] OpenRouter, *OpenRouter: Unified Interface for Large Language Models*, https://openrouter.ai/, Accessed: 2026-10-12, 2026.

[5] OpenRouter, *Cohere North Mini Code – OpenRouter Model Catalog*, https://openrouter.ai/cohere/north-mini-code:free, Accessed: 2026-10-12, 2026.

[6] GreyDGL, *PentestGPT: Automated Penetration Testing Agentic Framework Powered by Large Language Models*, https://github.com/GreyDGL/PentestGPT, Accessed: 2026-10-10, 2025.

[7] usestrix, *Strix: Open-source AI Penetration Testing Agents*, https://github.com/usestrix/strix, Accessed: 2026-10-10, 2025.

[8] KeygraphHQ, *Shannon: AI-Powered Penetration Testing Framework*, https://github.com/KeygraphHQ/shannon, Accessed: 2026-10-10, 2025.

[9] vxcontrol, *PentAGI: Autonomous Penetration Testing Orchestrator*, https://github.com/vxcontrol/pentagi, Accessed: 2026-10-10, 2025.

[10] BugTraceAI, *BugTraceAI-WEB: Autonomous AI-Powered Web Penetration Testing*, https://github.com/BugTraceAI/BugTraceAI-WEB, Accessed: 2026-10-10, 2026.

[11] Alias Robotics, *CAI: Cybersecurity AI Framework*, https://github.com/aliasrobotics/cai, Accessed: 2026-10-10, 2025.

[12] Ollama, *Ollama: Run AI Models Locally and in the Cloud*, https://ollama.com/, Accessed: 2026-10-12, 2026.

[13] Ollama, *FAQ – Ollama Documentation*, https://docs.ollama.com/faq, Accessed: 2026-10-12, 2026.

[14] Ollama, *Hardware Support – Ollama Documentation*, https://docs.ollama.com/gpu, Accessed: 2026-10-12, 2026.

[15] Ollama, *qwen2.5-coder:7b – Ollama Library*, https://ollama.com/library/qwen2.5-coder:7b, Accessed: 2026-10-12, 2026.

[16] Ollama, *qwen2.5-coder:14b – Ollama Library*, https://ollama.com/library/qwen2.5-coder:14b, Accessed: 2026-10-12, 2026.

[17] OWASP, *Running OWASP Juice Shop – Docker Container*, https://github.com/juice-shop/juice-shop/blob/master/README.md, Accessed: 2026-10-12, 2025.

[18] OWASP, *Running OWASP Juice Shop*, https://howto-llm.owasp-juice.shop/, Accessed: 2026-10-12, 2026.

[19] S. Sheng, J. Zhang, J. Yang, K. Cheng e Z. Qin, "PENTESTLLMAGENT: A Task Dependency Graph Planning-Based Multi-Agent Framework for Automated Penetration Testing," set. 2026, pp. 6518–6527. DOI: 10.24963/ijcai.2026/725